

TypeORM Fundamentals

Section	10 — TypeORM Introduction & Setup
Subtopic	10-01 — TypeORM Fundamentals
Duration	~2 hours (part of the 6–8 hr section)
Tags	Theory TypeScript
Topics covered	What is TypeORM, ORM concept and impedance mismatch, raw SQL vs ORM trade-offs, TypeORM vs Sequelize vs Prisma, supported databases

Learning Objectives — This Subtopic

- Explain what an ORM is and the impedance mismatch problem it solves
- Describe TypeORM's place in the Node.js ecosystem and why it is TypeScript-first
- Distinguish the trade-offs between writing raw SQL and using an ORM
- Compare TypeORM, Sequelize, and Prisma across key criteria to inform tool selection
- Identify which databases TypeORM supports and the implication for project portability

The Problem: Two Different Worlds

Relational databases organise data as tables of rows and columns, enforced by a strict schema. Object-oriented programming languages like TypeScript organise data as objects with properties and methods, connected by references. These two models work differently enough that moving data between them requires deliberate translation every time.

This tension is called the **object-relational impedance mismatch**. It manifests in four specific ways:

Mismatch	Database world	Object world
----------	----------------	--------------

Structure	Flat rows in a table with typed columns	Nested objects with arbitrary depth
Identity	A row is identified by a primary key value	An object is identified by its memory address
Relationships	Foreign keys + JOIN queries to link tables	Direct object references and nested arrays
Inheritance	No native concept; must be simulated with table strategies	Class hierarchies with extends and interfaces

What This Looks Like in Practice

Consider a `User` with a collection of `Post` records. In a relational database, these live in two separate tables linked by a foreign key. In TypeScript, you want a `User` object with a `posts` property containing an array of `Post` objects — ready to use, no further assembly required.

Without an ORM, every route that needs a user with their posts requires you to:

1. Write a `SELECT` to fetch the user row
2. Write a second `SELECT` filtered by the user's ID to fetch their posts
3. Manually assemble a TypeScript object from the raw column values
4. Handle `null` columns, type conversions (`DATE` strings to `Date` objects), and missing rows yourself

Multiply this by every entity in your application — users, products, orders, reviews — and the volume of repetitive, error-prone SQL assembly code becomes significant. An ORM automates this translation layer.

Analogy — Think of the database as a warehouse that stores everything in flat, labelled boxes on numbered shelves. Your application works with fully assembled products, not flat-packed components. An ORM is the assembly and disassembly line: it breaks your objects into flat boxes when saving, and reassembles them into fully formed objects when loading. You describe the shape of the product once; the assembly line handles the rest.

What is an ORM?

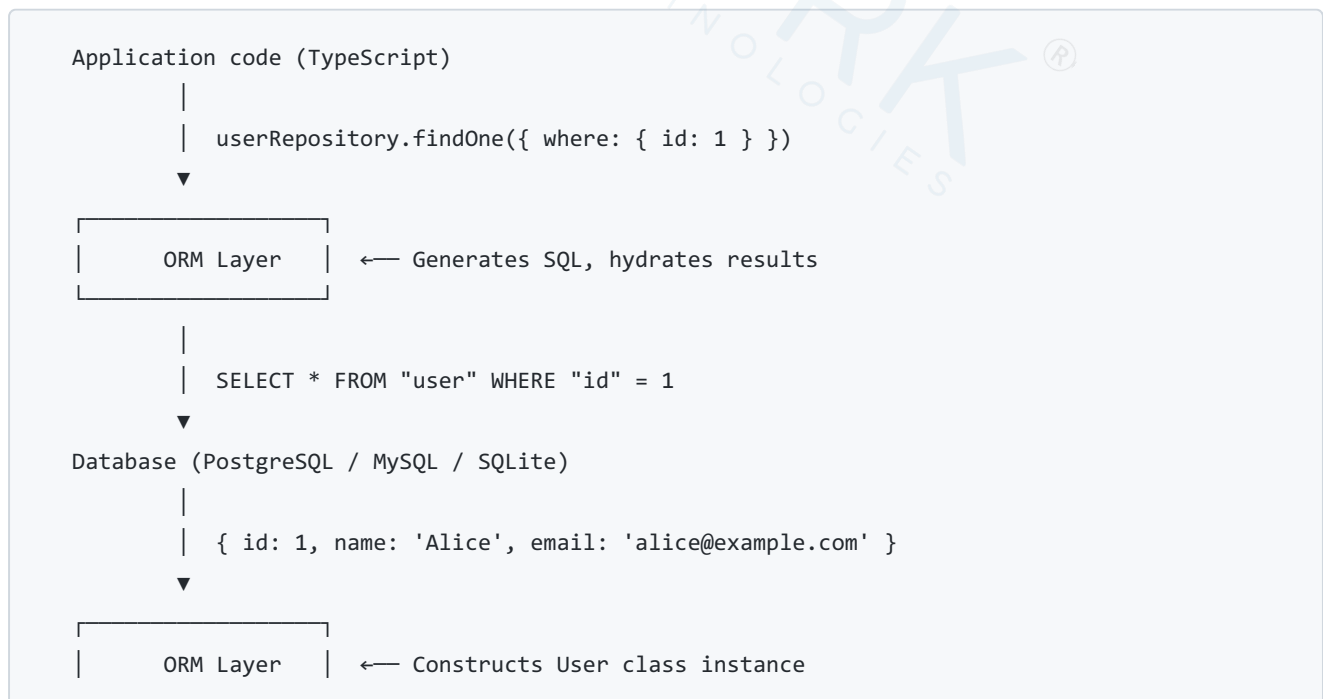
An **Object-Relational Mapper** is a library that creates a mapping between classes in your application code and tables in a relational database. It provides an API for querying and manipulating data entirely in the language of your application — TypeScript objects and method calls — without writing SQL directly in most cases.

The ORM handles the following automatically:

- **Schema mapping** — your class definition describes the table structure
- **Query generation** — method calls are translated into SQL statements
- **Result hydration** — raw database rows are converted back into typed class instances
- **Relationship loading** — related records from other tables are fetched and attached as nested objects
- **Type coercion** — database types (`TIMESTAMP` , `BOOLEAN` , `DECIMAL`) are converted to their TypeScript equivalents

The Conceptual Flow

At its core, an ORM sits between your application logic and the database driver, translating in both directions:



```
|  
|  
| User { id: 1, name: 'Alice', email: 'alice@example.com' }  
▼  
Application code receives a fully typed object
```

What is TypeORM?

TypeORM is an ORM for TypeScript and JavaScript that runs in Node.js. It was built from the ground up with TypeScript in mind, using TypeScript **decorators** as the primary mechanism for defining the mapping between classes and database tables. This makes entity definitions concise and co-located with the class code they describe.

TypeORM's key characteristics:

- **TypeScript-first** — decorators, generics, and type inference are central to its API
- **Active Record and Data Mapper patterns** — supports both; we use Data Mapper (repositories) in this module
- **Multiple database support** — the same entity code works across several database engines with minimal changes
- **Migration support** — generates SQL migration files to evolve your database schema safely over time
- **QueryBuilder** — a fluent API for constructing complex queries when the standard repository methods are insufficient

Note — Decorators and TypeScript: TypeORM makes heavy use of TypeScript decorators (`@Entity()`, `@Column()`, `@PrimaryGeneratedColumn()` and others). Decorators are a TypeScript feature (technically a stage-3 ECMAScript proposal) that attaches metadata to classes and their members. They require `"experimentalDecorators": true` in `tsconfig.json`, which we set up in Section 10-02. This is one of the primary reasons TypeORM projects use TypeScript rather than plain JavaScript.

A TypeORM entity is an ordinary TypeScript class annotated with decorators. The decorators tell TypeORM which table the class maps to, which properties are columns, and how relationships between entities are structured. The full entity API is covered in Section 11, but here is a preview of what the mapping looks like conceptually:

Forward reference — Section 11: The following snippet is shown for orientation only. Do not focus on the syntax at this stage — that is the subject of Sections 10-02, 10-03, and 11 in full. The purpose here is to establish *what TypeORM is doing* conceptually before working with it directly.

concept-preview.ts (not runnable yet – illustrative only)

```
import { Entity, PrimaryGeneratedColumn, Column } from 'typeorm';

@Entity() // ← maps this class to a database table
class User {
  @PrimaryGeneratedColumn() // ← auto-incrementing primary key column
  id: number;

  @Column() // ← maps this property to a table column
  name: string;

  @Column({ unique: true }) // ← column with a UNIQUE constraint
  email: string;
}
```

TypeORM reads the decorator metadata at runtime and knows: there is a table called `user`, it has three columns (`id`, `name`, `email`), and `email` has a unique constraint. All of this is expressed in TypeScript, next to the class it describes.

Raw SQL vs ORM: Trade-offs

Using an ORM is not universally superior to raw SQL — it is a trade-off. Understanding where each approach excels informs when to use which, and also when to bypass the ORM for a specific query even within a TypeORM project (which TypeORM explicitly supports via its `query()` method and `QueryBuilder` raw SQL escape hatch).

Raw SQL

- Full control over every query
- No abstraction overhead
- Easier to hand-tune performance
- Database-specific features freely available
- Steep learning curve for complex joins
- Manual type mapping and object assembly
- Schema changes require updating query strings throughout the codebase
- Risk of SQL injection if parameterisation is not applied consistently

vs

ORM (TypeORM)

- Entities and relationships declared once
- Automatic query generation for standard operations
- Type-safe result objects, no manual assembly
- Parameterised queries by default — injection protection built in
- Schema changes reflected via migration generation
- Abstraction can obscure what SQL is being executed
- Complex queries may be awkward to express; raw SQL escape hatch available
- Adds a dependency and a learning curve of its own

Key Insight: TypeORM does not prevent you from writing raw SQL. The `dataSource.query('SELECT ...', [params])` method executes arbitrary SQL directly. The practical approach for most applications is to use the ORM for standard CRUD operations and repository queries, and drop to raw SQL for reporting queries, bulk operations, or queries that require database-specific syntax.

When Raw SQL Still Makes Sense Inside a TypeORM Project

- Complex aggregations (`GROUP BY` , window functions) that are verbose to express with QueryBuilder
- Bulk inserts of thousands of records where ORM overhead per-row is measurable
- Database-specific features (PostgreSQL `JSONB` operators, MySQL `FULLTEXT` search)
- Read-only reporting queries that do not map to any entity

TypeORM vs Sequelize vs Prisma

The Node.js ecosystem has three dominant ORM / database-access libraries. Each makes different architectural choices, and you will encounter all three in professional codebases. Understanding the distinctions helps you read others' code and articulate why TypeORM is chosen for this module.

Criterion	TypeORM	Sequelize	Prisma
Primary language	TypeScript-first (JS works)	JavaScript-first (TS support added later)	TypeScript-first
Schema definition	TypeScript decorators on entity classes	JS/TS model definitions via <code>define()</code> or <code>Model.init()</code>	Separate <code>schema.prisma</code> file in Prisma's own DSL
Patterns supported	Active Record <i>and</i> Data Mapper	Active Record only	Data Mapper (via generated Prisma Client)
Migrations	Generated from entity changes; run via CLI	Handwritten migration files	Generated from schema changes; run via CLI
Type safety	Good — decorators provide types; some gaps	Moderate — required extra configuration historically	Excellent — fully generated types from schema; zero-gap
Query API	Repository methods + QueryBuilder fluent API	Model static methods + raw <code>query()</code>	Prisma Client (auto-generated, fully typed)
Supported databases	MySQL, PostgreSQL, SQLite, MS SQL, Oracle, MongoDB (experimental)	MySQL, PostgreSQL, SQLite, MS SQL, MariaDB	MySQL, PostgreSQL, SQLite, MS SQL, MongoDB, CockroachDB
Learning curve			

	Moderate — decorators, DataSource config, QueryBuilder	Moderate — large surface area, inconsistencies in docs	Distinct — must learn Prisma schema DSL and workflow
Maturity / adoption	Established; widely used in NestJS ecosystem	Oldest and most battle-tested; large community	Rapidly growing; favoured in newer greenfield projects

Why TypeORM for This Module?

TypeORM is the natural fit here for two specific reasons. First, its decorator-based entity definition sits directly on TypeScript classes — the mapping is readable and co-located with the code it describes, which makes it highly teachable. Second, TypeORM is the default ORM in **NestJS**, the most widely adopted TypeScript server framework in professional Node.js development. The patterns you learn here — entities, repositories, DataSource configuration, migrations — transfer directly to NestJS projects.

Note — Prisma's trade-off: Prisma's type safety is genuinely superior, and it is worth exploring after this module. Its main trade-off is a separate schema file in an external DSL, which divorces the data model from the TypeScript code. TypeORM keeps everything in TypeScript. Neither approach is universally correct; the choice depends on team preference and project requirements.

Supported Databases

TypeORM abstracts the differences between database engines behind a unified API. Switching databases in a TypeORM project ideally requires only a configuration change — the entity definitions and repository calls remain identical. In practice, some SQL dialects differ enough that a few queries may need adjustment, but standard CRUD operations and relationships are fully portable.

Database	Driver package	Notes
PostgreSQL	pg	

		Recommended for production. Strongest support for TypeORM's full feature set including JSONB columns and advanced index types.
MySQL / MariaDB	<code>mysql2</code>	Widely used in shared hosting environments. <code>mysql2</code> is the modern driver; the older <code>mysql</code> package is deprecated for TypeORM use.
SQLite	<code>better-sqlite3</code> or <code>sqlite3</code>	File-based database — no separate server process required. Ideal for local development, testing, and embedded applications. Used in this module's practicals for zero-setup convenience.
Microsoft SQL Server	<code>mssql</code>	Common in enterprise Windows environments. Supported but less commonly used in the Node.js ecosystem.
Oracle	<code>oracledb</code>	Supported but requires Oracle Instant Client. Rarely used in new projects.
MongoDB	<code>mongodb</code>	Experimental support. TypeORM's relational model maps poorly to MongoDB's document model. Mongoose is the standard choice for MongoDB in Node.js.

Why SQLite for Practicals

Throughout Sections 10–14, practicals use SQLite. SQLite requires no installation of a database server, stores the entire database in a single file on disk, and is automatically created when the application first runs. This eliminates the environment setup variability that would otherwise consume a significant portion of practical time.

Important: SQLite is not suitable for production web applications. It does not support concurrent writes from multiple processes, which rules it out for any server that handles simultaneous requests writing to the database. Production deployments use PostgreSQL or MySQL. The TypeORM entity and repository code you write for SQLite practicals is *identical* to what you would use in production with PostgreSQL — only the DataSource configuration changes.

The Portability Benefit

Database portability is one of the ORM's concrete benefits in real projects. A common workflow is:

- **Development** — SQLite (no server, instant setup, file deleted and recreated freely)
- **CI / automated tests** — SQLite or a PostgreSQL container
- **Staging and production** — PostgreSQL or MySQL

With TypeORM, the only difference between these environments is the `DataSource` configuration object — covered in full in Section 10-02.

Two Patterns: Active Record and Data Mapper

TypeORM supports two architectural patterns for interacting with the database. This module uses **Data Mapper** exclusively, but you need to recognise both when reading documentation and others' code.

Active Record

In the Active Record pattern, the entity class itself contains the database access methods. A `User` class has a `User.find()` method, a `user.save()` method, and so on. The entity is both the data structure and the data-access layer.

Forward reference — Section 11: The snippet below is conceptual illustration only.

active-record-concept.ts (illustrative only)

```
// Active Record: the entity class manages its own persistence
const user = new User();
user.name = 'Alice';
await user.save(); // saves directly from the entity instance

const found = await User.findOneBy({ id: 1 }); // static method on the class
```

Data Mapper

In the Data Mapper pattern, the entity class is a plain data container — it has no knowledge of the database. A separate **repository** object handles all database operations. The entity and the data-access logic are cleanly separated.

data-mapper-concept.ts (illustrative only)

```
// Data Mapper: a separate repository manages persistence
const userRepository = dataSource.getRepository(User);

const user = new User();
user.name = 'Alice';
await userRepository.save(user); // repository handles the save

const found = await userRepository.findOneBy({ id: 1 });
```

Why Data Mapper for This Module

Criterion	Active Record	Data Mapper
Entity classes	Aware of the database; contain save/find methods	Plain data containers; no database knowledge
Testability	Harder to test — database coupling in the entity	Easier — entity can be instantiated without a database connection

Separation of concerns	Data and persistence logic mixed	Clear separation between domain model and data access
NestJS compatibility	Not the idiomatic NestJS approach	Standard pattern in NestJS TypeORM integration

Data Mapper is also the pattern used by TypeORM's own documentation for advanced usage, and it aligns directly with the Repository pattern covered in Section 11. All practicals from Section 10 onwards use repositories.

Summary

Concept	Key Point
Impedance mismatch	The structural gap between relational tables (flat rows) and OO objects (nested, reference-linked) that must be bridged every time data moves between the database and the application
ORM	A library that maps classes to tables, generates SQL from method calls, and hydrates raw rows back into typed objects — automating the translation layer
TypeORM	A TypeScript-first ORM for Node.js that uses decorators on entity classes to define the database mapping; supports both Active Record and Data Mapper patterns
Raw SQL vs ORM	ORM trades some query flexibility and transparency for type safety, automatic hydration, and reduced boilerplate; TypeORM supports raw SQL as an escape hatch
TypeORM vs Sequelize	

	TypeORM is decorator/TypeScript-first; Sequelize is older and JS-first with weaker native TS support
TypeORM vs Prisma	Prisma has stronger generated type safety but uses an external schema DSL; TypeORM keeps everything in TypeScript decorators
Supported databases	PostgreSQL (recommended production), MySQL/MariaDB, SQLite (practicals), MS SQL, Oracle; MongoDB support is experimental
Active Record	Entity class manages its own persistence via static/instance methods — entity is aware of the database
Data Mapper	Entity is a plain data container; a separate repository object handles all database operations — separation of concerns, easier to test; the pattern used in this module

Interview Questions

1. What is the object-relational impedance mismatch? Give two concrete examples of where it occurs.

Expected answer: The impedance mismatch is the structural incompatibility between relational databases (tables of typed rows) and object-oriented code (nested objects connected by references). Two examples: (1) *Relationships* — in the database, a user's posts are a separate table linked by foreign key, requiring a JOIN to retrieve; in code, you want `user.posts` as a ready-to-use array. (2) *Inheritance* — a class hierarchy (`Animal` → `Dog`) has no direct equivalent in relational tables; it must be simulated using strategies like single-table inheritance or joined-table inheritance.

2. What does "hydration" mean in the context of an ORM?

Expected answer: Hydration is the process of taking raw data — typically a flat array of column values returned by the database driver — and populating (inflating) a typed class instance with those values. The result is a fully formed object rather than an anonymous row. TypeORM hydrates results automatically when using repositories or QueryBuilder with an entity target.

3. A colleague argues that ORMs should always be avoided because they hide what SQL is being executed. How would you respond?

Expected answer: The concern is valid — opaque query generation can produce inefficient SQL, and understanding what hits the database is important for performance. However, the argument for avoiding ORMs entirely ignores the significant developer productivity gains: automatic type mapping, safe parameterised queries by default, and co-located schema definitions. The pragmatic approach is to use the ORM for standard operations while monitoring generated queries (TypeORM logs them when `logging: true` is set in DataSource config) and dropping to raw SQL when the ORM's generated query is suboptimal.

4. What is the difference between the Active Record and Data Mapper patterns in TypeORM? Which does this module use, and why?

Expected answer: In Active Record, the entity class itself has database access methods (`user.save()` , `User.find()`). In Data Mapper, the entity is a plain data container with no database knowledge; a separate repository object (`userRepository.save(user)`) handles persistence. This module uses Data Mapper because it cleanly separates domain logic from data access, makes entities independently testable, and is the standard pattern in NestJS TypeORM integration.

5. Why does TypeORM require TypeScript's experimental decorators, and what configuration enables them?

Expected answer: TypeORM uses TypeScript decorators (`@Entity()` , `@Column()` , etc.) to attach metadata to classes at definition time. TypeScript decorators are not yet finalised in the ECMAScript standard, so they require the `"experimentalDecorators": true` flag in `tsconfig.json` to compile. Additionally, `"emitDecoratorMetadata": true` is typically needed so TypeORM can read reflected type information at runtime.

6. Compare TypeORM and Prisma on the dimension of type safety. What is Prisma's advantage, and what is its trade-off?

Expected answer: Prisma generates a fully typed client from its schema file, meaning every query result is precisely typed with zero manual annotation — the types are guaranteed to

match the database schema. TypeORM's types come from decorator-annotated classes, which is generally good but has some gaps (e.g., nullable columns require manual handling). Prisma's trade-off is that its schema is written in a proprietary DSL (`schema.prisma`), which is a separate file and a separate language to learn; TypeORM keeps everything in TypeScript.

7. Why is SQLite used in practicals if it is not suitable for production?

Expected answer: SQLite requires no server installation — it stores the database in a single file and creates it automatically on first run. This eliminates environment setup variability across different machines during training. The entity definitions and repository calls written for SQLite are identical to those used with PostgreSQL in production; only the DataSource configuration object changes. SQLite's unsuitability for production (no concurrent write support) is a deployment concern, not an API concern.

8. You are starting a new Node.js project and must choose between TypeORM, Sequelize, and Prisma. What questions would you ask to inform the decision?

Expected answer: Key questions include: Is the team using TypeScript (Prisma and TypeORM both excel here; Sequelize's TS support is more effort)? Will the project use NestJS (TypeORM is the natural choice)? Is maximum type safety a priority and is the team comfortable with a separate schema DSL (Prisma)? Is the codebase joining an existing ecosystem where one ORM is already established? Are there plans to switch databases — all three support multiple engines, but Sequelize has the longest track record with MySQL/PostgreSQL in particular.

Practical Assignment — Mapping the Mismatch

Objective: This is a deliberate *non-code* practical. Before writing a single line of TypeORM, you must be able to articulate the impedance mismatch problem and the ORM's solution in writing. This reinforces conceptual understanding before the technical detail of Sections 10-02 and 11.

Part A — Identify the Mismatch (written exercise)

Consider the following domain: an online bookstore with these entities and relationships:

- A **Book** has a title, an ISBN, a price, and a publication date
- A **Book** belongs to one **Author**

- An **Author** has many **Books**
- A **Book** belongs to many **Categories** (e.g., Fiction, Mystery)
- A **Category** contains many **Books**

Write answers to the following:

1. Draw or describe the **relational schema** for this domain: list the tables, their columns, primary keys, and foreign keys. How many tables are needed in total?
2. Draw or describe the **TypeScript class structure** you would want in your application: list the classes, their properties, and how they reference each other.
3. Identify **three specific points of mismatch** between your relational schema and your TypeScript class structure.
4. For each mismatch, describe in one sentence what the ORM must do automatically to bridge it.

Part B — ORM Comparison Matrix (research exercise)

Look up the documentation for Sequelize (sequelize.org) and compare how it defines a model (equivalent to a TypeORM entity) to how TypeORM defines one with decorators. Produce a side-by-side code comparison showing both approaches for the same `Book` entity with at minimum: primary key, title (string), price (decimal), and publication date (date). Write one paragraph summarising the structural difference between the two approaches.

Output:

Part A expected schema:

```
Tables: book, author, category, book_category (junction table)
book: id PK, title, isbn, price, publication_date, author_id FK → author.id
author: id PK, name, biography
category: id PK, name
book_category: book_id FK → book.id, category_id FK → category.id
```

TypeScript classes:

```
class Book { id, title, isbn, price, publicationDate: Date, author: Author, categories: Category[] }
class Author { id, name, biography, books: Book[] }
class Category { id, name, books: Book[] }
```

Mismatches to identify (examples):

1. `book_category` junction table has no TypeScript class equivalent – the ORM hides it behind arrays
2. `author_id` FK column in `book` table → TypeScript uses a full `Author` object reference instead
3. `publication_date` stored as `DATE` string → TypeScript wants a `Date` object (type coercion needed)

Bonus Challenge

Research TypeORM's **inheritance strategies**: Single Table Inheritance and Class Table Inheritance. For a domain with a `Vehicle` base class and `Car` and `Truck` subclasses, describe in one paragraph how each strategy represents the hierarchy in the database, and what the trade-off between them is. You will encounter these decorators in Section 11.

Notes on content additions/changes from syllabus: The syllabus lists four bullet points for this subtopic. This document adds: (1) a dedicated "two different worlds" framing section that establishes the impedance mismatch concretely via a table of four specific mismatches, rather than naming it abstractly — necessary groundwork for the ORM to make sense; (2) a conceptual flow diagram showing the ORM's position in the call chain between application and database; (3) the Active Record vs Data Mapper section — not listed in the 10-01 bullets but essential context before students encounter repository-style code in 10-02 and Section 11; it is introduced here conceptually so the pattern decision is understood before configuration begins; (4) the practical assignment is intentionally code-free — this is the only section where that is appropriate, as all subsequent sections are hands-on; a written mismatch mapping exercise builds the mental model that makes decorator syntax intuitive rather than magical when students first see it in Section 11.